

Projet ETML-ES – Modification POBJ

PROJET:	1730 Portier Wireless		
Entreprise/Client:	ETML-ES	Département:	SLO
Demandé par (Prénom, Nom):	PBY	Date:	03.02.2025
Objet (No ou réf, pièce, PCB...)	20200214_Portier_17300B_SCA		
Version à modifier:	V2.1		

Auteur (ETML-ES):	Léo Mendes	Filière:	SLO
Nouvelle version:	V3.0	Date:	03.02.2025

1 Projet

1.1 Description sommaire du projet à modifier

Ce projet consiste en la conception et la réalisation d'un portier sans fil basé sur un microcontrôleur PIC32MX130F064B-I/SS et un module radio 868 MHz. Le système est composé de deux circuits distincts :

- Système OUT (extérieur) : placé à l'entrée du bureau, il est alimenté par deux piles AAA, assurant une autonomie de plusieurs mois. Il permet à un utilisateur d'envoyer une requête vers l'intérieur en appuyant sur un bouton.
- Système IN (intérieur) : installé dans le bureau, il est alimenté via un port micro-USB (5V) avec un régulateur 3.3V. Il reçoit la requête et permet à l'occupant de répondre par différents signaux lumineux et sonores.

Le fonctionnement repose sur une communication UART à 57'600 bauds entre les modules RF des circuits IN et OUT. Un système d'auto-alimentation est intégré au circuit OUT pour optimiser la consommation énergétique. De plus, une mesure de tension des piles est implémentée pour avertir lorsque leur remplacement est nécessaire.

L'objectif du projet est d'obtenir une solution robuste, efficace et peu énergivore, répondant aux besoins d'une communication simple et rapide entre un visiteur et un occupant d'un bureau.

1.2 Référence conception

Projet stocké en interne :

K:\ES\PROJETS\SLO\1730_PortierWireless\doc

Repo Git du projet :

https://github.com/ETML-ES-SLO/PRJ_1730_PortierWireless/tree/LMS

2 Objectifs principaux des modifications ou ajouts

Plusieurs modifications sont demandées :

2.1 Algorithme - appairage entre différents modules

L'identification des modules est partiellement mise en place grâce à la gestion des numéros de série. Actuellement, le programme peut lire et comparer les numéros de série des modules pour vérifier s'ils sont enregistrés. Cependant, il manque un véritable mécanisme d'appairage dynamique. Les fonctions `SaveSerialInFlash()`, `RemoveSerial()` et `ResetSerialList()` sont définies mais vides, ce qui empêche l'ajout ou la suppression d'un module directement depuis le système.

Pour finaliser l'appairage, il faut implémenter ces fonctions afin de permettre l'enregistrement et la gestion dynamique des modules autorisés. Une méthode d'enregistrement automatique lors de la première utilisation pourrait aussi être ajoutée pour simplifier le processus.

2.2 Fonction envoi msg -> module alimenté pour changement de pile

La tension de la batterie est actuellement intégrée dans les trames RF (bbb en centièmes de volt), et l'unité intérieure peut détecter une batterie faible en utilisant `appData.isLowBat`. Cependant, aucun message spécifique n'est envoyé au module intérieur pour signaler explicitement un besoin de remplacement des piles.

Pour compléter cette fonctionnalité, il faudrait ajouter un message RF dédié ([D00LOW] par exemple) lorsqu'une batterie atteint un seuil critique. L'unité intérieure pourrait alors afficher une alerte ou enregistrer cet événement pour notification ultérieure.

2.3 Fonction sonore et visuelle pour changement de la batterie

L'indication visuelle est partiellement implémentée. Lorsque la batterie est faible, les LED clignotent rapidement, mais aucun voyant dédié ne permet de différencier cet état d'autres alertes. Actuellement, le clignotement rapide des trois LED (`LED_ID_ENTER`, `LED_ID_WAIT`, `LED_ID_BUSY`) est utilisé pour alerter sur un niveau de batterie bas.

Concernant l'alerte sonore, rien n'est prévu dans le programme. Il n'existe pas de mélodie spécifique pour signaler une batterie faible, et les sons déjà implémentés (`RING`, `SUCCESS`, `ERROR`) sont dédiés à d'autres fonctions. Il serait pertinent d'ajouter un son court et distinctif pour avertir de la nécessité de changer les piles. Une modification de `Bell_Mgmt()` permettrait de déclencher ce signal sonore dès qu'une batterie faible est détectée.

3 Résumé de l'état actuel

Le système de portier sans fil fonctionne sur un PIC32MX130F064B-I/SS et utilise un module RF 868MHz pour communiquer entre une unité extérieure (porte) et une unité intérieure (sonnette). Il est capable d'échanger des trames RF, de vérifier les numéros de série pour l'identification des modules, et de gérer les interactions utilisateur via des LED et un haut-parleur.

L'unité extérieure envoie une trame lorsqu'un bouton est pressé. L'unité intérieure peut alors répondre avec un message (E = Entrez, W = Attendez, B = Occupé, N = Personne). La tension de la batterie de l'unité extérieure est intégrée dans les messages envoyés, permettant à l'unité intérieure de détecter un niveau de charge faible.

Les LED permettent d'indiquer l'état du système, avec différentes configurations (allumé, éteint, clignotement normal ou rapide). Le haut-parleur joue des sons spécifiques pour différentes actions, comme la confirmation d'entrée ou un signal d'erreur.

4 Tâches à réaliser

Le livrable final sera un firmware mis à jour pour le portier sans fil basé sur le PIC32MX130F064B-I/SS, intégrant les améliorations demandées. Cette nouvelle version inclura les corrections et ajouts nécessaires pour compléter les fonctionnalités inachevées, tout en respectant le standard ETM-ES en termes de conception, codage et validation.

Le livrable comprendra :

4.1 Firmware V3.0:

- Ajout d'un algorithme complet d'appairage permettant d'enregistrer et de supprimer dynamiquement des modules.
- Implémentation d'un message RF dédié pour signaler un niveau de batterie critique à l'unité intérieure.
- Ajout d'un signal sonore spécifique pour alerter un utilisateur d'un besoin de changement de piles.

4.2 Documentation technique mise à jour :

- Flowchart des nouvelles fonctionnalités.
- Machines d'état révisées pour intégrer les nouvelles transitions et événements.
- Liste des fonctions modifiée.

4.3 Rapport de tests et validation :

- Test d'implémentations

4.4 Table des tâches à réaliser

Contenu du tableau ci-dessous en exemple

#	Priorité	Description	Fait	Vu
1	1	Lire le rapport effectué par Ismaël Page	LMS	OK
2	1	Lire modifications de Diogo Ferreira	LMS	OK
4	1	Reprogrammez et faire fonctionner projet en état actuelle		
5	1	Compréhension et vue d'ensemble du code et des objectifs		
6	2	Mise en place des modifications : 2.1Algorithme - appairage entre différents modules		
7	2	Mise en place des modifications : 2.2Fonction envoi msg -> module alimenté pour changement de pile		
8	2	Mise en place des modifications : 2.3Fonction sonore et visuelle pour changement de la batterie		
9	2	Test des fonctions implémentées		
n	1	Documenter tout le travail effectué.		

1 = Elevé

2 = Moyen

3 = Faible

5 Détail des modifications ou ajouts

#	Description	Fait	Approuvé
1	Implémentation complète de l'appairage dynamique des modules - Finalisation des fonctions SaveSerialInFlash(), RemoveSerial(), ResetSerialList() pour gérer l'enregistrement et la suppression des numéros de série en mémoire Flash - Ajout d'une méthode d'appairage automatique lors de la première utilisation pour simplifier le processus. - Vérification et validation des numéros de série avant enregistrement pour éviter les doublons ou erreurs de stockage.		
2	Mise en place d'un message RF dédié pour alerter sur un niveau de batterie critique - Ajout d'un message [D00LOW] envoyé automatiquement lorsque la batterie atteint un seuil critique. - Modification du firmware pour que l'unité intérieure affiche une alerte visuelle et/ou sonore en cas de réception de ce message. - Possibilité d'enregistrement de l'alerte pour consultation ultérieure.		
3	Amélioration de l'indication visuelle en cas de batterie faible - Ajout d'un voyant dédié (ou clignotement distinct) pour signaler un état de batterie faible, différent des autres alertes système. - Modification du comportement des LED pour éviter toute confusion avec d'autres états d'alerte.		
4	Implémentation d'une alerte sonore spécifique pour signaler une batterie faible. - Ajout d'un son court et distinctif déclenché dès qu'une batterie faible est détectée. - Intégration de cette alerte dans Bell_Mgmt() pour garantir une activation automatique sans intervention utilisateur. - Vérification que ce signal sonore ne se superpose pas avec d'autres notifications sonores déjà existantes.		
5	Mise à jour de la documentation technique avec Flowchart, machines d'état et tests		

6 Matériel nécessaire

- MPLAB X version IDE et version compilateur
- Projet 17300C avec récepteur et émetteur.
- Débugueur SNAP.

6.1 Stockage du fichier

Ce fichier sera stocké « K:\ES\PROJETS\SLO\1730_PortierWireless\doc »

Ainsi, tous les fichiers de modifications des pièces ou PCBs faisant partie du projet sont centralisés dans le même répertoire. La numérotation devient implicite.

6.2 Stockage des logiciels

Les codes sources sont stockés ici :

« K:\ES\PROJETS\SLO\1730_PortierWireless\soft »

https://github.com/ETML-ES-SLO/PRJ_1730_PortierWireless/tree/LMS